202323017 – TLALI THUTO

Moderated Learning Management System (LMS)

C++17 and SQLite Implementation

Project: Moderated Learning Management System

Official Release: version 1.1

Tools: C++17, SQLite, CMake and GoogleTest

Project Introduction

The Moderated Learning Management System (LMS) is a software application developed to manage academic activities, including users, departments, courses, assignments, submissions, marking, moderation and the release of student results.

Educational institutions require reliable systems to manage assessment processes. Manual administration can result in inconsistent records, missed deadlines, calculation errors and limited accountability when marks are changed.

This project addresses these challenges by providing a system that supports academic administration and assessment.

Project Objectives

The main objectives were to:

- Develop an application using C++17 and object-oriented programming.
- Implement four user roles with different responsibilities.
- Store academic information using an SQLite database.
- Support assignment creation, submission and resubmission.
- Apply consistent late-submission penalties.
- Provide marking, feedback and moderation functionality.
- Record important assessment changes through audit logs.
- Test the system using automated testing.
- Implement Phase 8 maintenance requirements and release version 1.1.

Tool use

Tools	Purpose
C++17	Application development

SQLite	Database storage
CMake	Compilation, Linking and Building
GoogleTest	Automated testing
Git and GitHub	Application housing and code control
SQL	Database operations

These technologies provide a foundation for a local application that can be built, tested and maintained

System Design and Implementation

System Architecture

The LMS follows a layered architecture that separates responsibilities.

The **role layer** contains the Administrator, Lecturer, Head of Department (HoD) and Student classes.

The **service layer** implements business rules. Examples include AuthService, SubmissionService, ModerationService and AuditService.

The **repository layer** handles data storage and retrieval through repositories such as UserRepo, CourseRepo, AssignRepository, SubmissionRepo and AuditLogRepo.

The **database layer** uses the `Database` class to manage SQLite operations, parameterised queries and transactions.

This separation makes the application easier to understand, test and maintain.

Object-Oriented Design

The system applies object-oriented programming principles.

Inheritance allows specialised user roles to share common functionality through a base User class. Encapsulation groups related data and operations, while polymorphism supports specialised role behaviour through common interfaces.

Services and repositories also demonstrate separation of concerns. For example, SubmissionService enforces submission rules, while SubmissionRepo handles submission persistence.

Database Design

SQLite stores the system's academic information. The database includes tables for users, departments, courses, enrolments, assignments, extensions, submissions and audit logs.

Foreign keys maintain relationships between records, while parameterised SQL statements help protect against SQL injection.

The Phase 8 release adds a separate ClassExtensions table while retaining the existing individual extensions table.

Users' Main Roles

- **Administrator:** Manages users, departments and courses.
- **Lecturer:** Creates assignments, manages extensions, marks submissions and provides feedback.
- **HoD:** Moderates marks and approves official result releases.
- **Student:** Views courses and assignments, submits work and accesses feedback and released results.

Overall Functionality

Assessment Workflow

The assessment process follows these stages:

1. The Lecturer creates an assignment and specifies its deadline and maximum mark.
2. The Student views the assignment and submits work.
3. The system validates the submission and determines its lateness.
4. The Lecturer awards a mark and provides feedback.
5. The HoD reviews and moderates the mark.
6. The HoD approves the official result release.
7. The Student views the released result and feedback.

Submission and Late Penalties

The system records submission dates and statuses. Resubmission is permitted only when the applicable rules allow it. For example, replacing a submission after marking is restricted.

The late-penalty rule deducts 10% per late day, up to three days. Submissions more than three days late receive no credit under the configured rules.

This ensures that late penalties are calculated consistently.

Marking, Moderation and Audit Logging

Lecturers enter marks and written feedback. The system validates marks against the permitted range.

When an existing mark is changed, a reason is required. Audit records capture the responsible user, time, previous value, new value and reason.

The HoD provides an additional level of academic oversight by reviewing and moderating marks before official release.

Database transactions help ensure that important mark changes and their corresponding audit records are handled together.

Authentication and Data Protection

The system includes password hashing, role-based functionality, parameterised database queries and foreign-key constraints.

These mechanisms help protect user credentials, restrict responsibilities and maintain database integrity.

Deadline Rules

The system applies the following precedence:

1. **Individual extension:** Takes priority for the specific student.
2. **Class-wide extension:** Applies when no individual extension exists.
3. **Original deadline:** Applies when neither extension exists.

This prevents class-wide extensions from overriding individual arrangements.

The Student assignment list displays the effective deadline and extension type so that students can identify which deadline applies to them.

Version 1.1

Release v1.1 introduces the class-wide extension enhancement while retaining existing individual-extension functionality.

The maintenance deliverables include:

- Change request classification and impact analysis.
- Updated source code and database functionality.

- Regression-testing coverage.
- Version 1.1 release notes.

Conclusion

The Moderated Learning Management System was developed using C++17 and SQLite to support academic administration and assessment.

It provides functionality for managing users, courses, assignments, submissions, marking, moderation and official result releases.

Phase 8 introduced class-wide deadline extensions while preserving individual extensions. The enhancement applies a clear precedence rule and improves deadline visibility for students.

The project demonstrates the importance of software design, implementation, testing, documentation and maintenance. Although the Phase 8 enhancement and release documentation have been prepared, final regression verification remains necessary before the application can be declared fully tested.

Overall, the LMS provides a foundation for a more organised and accountable academic assessment process.